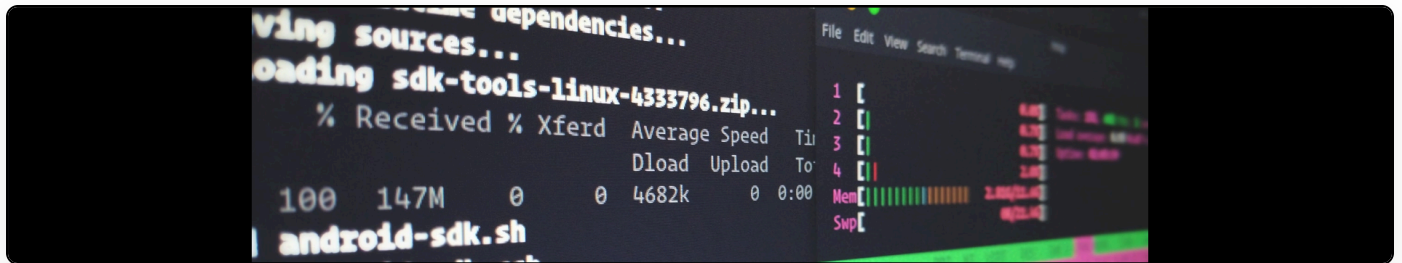


Evaluating the Open Access software toolchain

Cite as: Martin Paul Eve, "Evaluating the Open Access software toolchain", *eve.gd: Martin Paul Eve*, July 07, 2013, <https://doi.org/10.59348/1qjzt-kfe68>.



I received an interesting email this week from [Nate Wright](#), who posed the following questions:

I'm a web developer interested in contributing to a low-cost, open-source solution for online academic publishing. Prompted by a conversation with a former lecturer of mine, I've spent some time investigating the various open-source or low-cost options for digital journal publication (OJS, Scholastica, Annotum, Faculty, and the collection of tools being developed by the team at eLife). It looks like OJS is the only open-source platform out there which can provide end-to-end capabilities for running a journal. In my own experience, though, I've grown wary of niche CMS's, which lack a large body of tools and community support to help inexperienced site admins easily customise and extend their website. Even fairly large and well-maintained CMS's, like Silverstripe, really suffer from the small size of their community developing plugins and themes. OJS seems pretty tightly bound to traditional publishing cycles as well, which will limit its utility as academic publishing transitions to new models. Speaking purely from the perspective of a mainstream web developer, if I was advising someone setting up a journal now, I would tell them they were taking a risk by committing to OJS. It's not clear how a successful journal website could mature on the platform over time and whether or not data would be portable if (when) a better solution arises in the future.

Both Scholastica and Faculty look like promising, affordable, easy-to-use end-to-end platforms. I'm sure they'll offer a workable solution for many small-scale journals even though they're proprietary. But what I'm interested in contributing to is an open-source solution, built on a proven platform such as Wordpress or Drupal, which will ensure that content is "future-safe" and easy to customise, extend and adapt as academic publishing conventions change. These platforms are also supported by some of the cheapest hosting and design shops out there -- an unfortunate reality given the low budgets for open access journals in the humanities and social sciences.

This leaves Annotum for Wordpress. It looks like it has admirably managed to integrate some basic features -- JATS XML output, citation management and basic peer review. This is probably where I should put my efforts. But it was funded by Google so that Knol users could easily transfer their data. That's led the Annotum project to build everything into a bloated theme rather than modular plugins. It makes sense for their purposes, but it raises some concerns about the sustainability of the project.

Perhaps I'm being overly picky. But as a web developer I've come to feel that foundations really matter, because the web changes quickly and all-in-one tools can rarely keep up unless an organisation can afford regular, expensive development costs.

In order to make good choices about where I put my limited time, I'd like to better understand the core functionality needed to run a journal, and the priorities behind the toolsets that are needed. I was hoping you could help me by addressing a few questions. What are the tools you need to run a journal? Which tools need to be integrated with your online publishing platform? Which could be externalised to tools which are not tied into a particular publishing platform? What tools are you already using -- open-source or otherwise -- to meet your needs?

I asked Nate if he'd mind if I replied publicly to this in a blog post because, quite frankly, this issue is important:

- Although we always go by the aphorism that the social problems are the ones that need fixing, we cannot neglect the technological
- If we do not build and maintain an open toolset, we cannot rely on the arguments derived from the free software movement for ethical imperatives to OA
- If we do not build and maintain an open toolset, we will be beholden to proprietary lock-in and outside determination of workflow (which drives peer review)

So, let me sum up my position. OJS is an amazing piece of kit, but the criticism's above are entirely valid. OJS was ahead of the curve and developed along traditional workflow lines. I'm sure that PKP realise this. After all, Open Monograph Press has a far more flexible workflow (and awesomely sleek design!) Now, OJS is problematic for these reasons, but it's also amazing for running a journal and very much open to external contributions. It's GPL licensed, stored on GitHub and generally maintained by nice people who are willing to spend their time guiding new developers through their first commit.

Annotum does, indeed, look as though it's doing some amazing things, but the elephant in the room is [Ambra](#), PLOS' system. Now, I've heard mixed reports on Ambra ("is it overkill?" etc) and I can't comment on their policy towards collaborative development, but it is an Apache licensed project with a comprehensive Wiki and Trac system in place. Definitely worth exploring.

[Our](#) submission platform is going to take Ambra as a base, most likely. Typesetting in NLM format will be done through our (very) in-development tool, [meTypeset](#). This is AGPL v2 licensed and I'd welcome contributions; it's a basic attempt to start towards the functionality of extyles. The basic premise is that, through a series of XSLT stylesheets and python-driven regex parsing, it converts a Word/OpenOffice document into valid NLM/JATS XML. At present I also have a very rudimentary [citation parsing engine](#) in the works, also, as part of that project.

For my current layout generation, I use another in-house (but GPLed) tool, [meXmlGalley](#). This is derived from OJS' (aborted?) attempt to integrate NLM. I revived the project and got OJS to drive FOP once more, adapted the stylesheet to give palatable output and have been tweaking the layout since. However, I don't tend to drive it via OJS now (even on OJS-run journals), I run the bash script to produce the PDF and HTML galleys and just upload them. This saves the potential instability of using PHP to launch command line tools to re-generate the PDF on the fly.

In any case, let me sum up what I do:

- Use OJS for my small niche journal, [Orbit](#), which does a good job of document handling
- Use Wordpress, without Annotum, for the quasi-magazine/journal [Alluvium](#), which is great at looking good
- Use my own tools, [meTypeset](#) and [meXmlGalley](#) for typesetting and layout editing, currently with too much human intervention
- Use an Ambra testbed for experimentation

Here's what needs to happen (and I'm working on it):

The formation of an Open Access Toolset Alliance. I've begun to coordinate a group of people interested in this. The idea would be that we discuss what we are doing and ensure that we don't replicate labour re-building the same tools. There is scope for a variety of approaches, but if we are going to re-build the publishing toolchain in fully open software, we need to work in greater dialogue than the closed silos that can sometimes develop. There'll be a website forthcoming on this, but if people are interested, please [email me](#).

I think, to respond to Nate's specific question, that if we want tools that can plug in to any architecture, then we need to start working out where we have tied things too closely to our platforms, come up with standard interface formats and begin abstracting the functionality. In fact, that might be the best approach: find a project that has locked in the functionality and to then liberate it might be a good first step.